

NoSQL Product Catalog Design

E-Commerce Order Management Database System | Skill Development Activity (DBMS)

1. Objective

Design a NoSQL representation of the product catalog using JSON documents and key-value structures, and explain how it relates to the relational design finalized in Week 6.

2. Why a Catalog Suits NoSQL

In the relational design, product attributes are fixed columns. Real catalogs are not uniform: a phone has `ram` and `battery`, a T-shirt has `size` and `fabric`, and a book has `author` and `isbn`. In SQL this leads to many NULL columns or an extra attribute table. A document database such as MongoDB lets each product carry exactly the attributes it needs, and a product page can be loaded with one read instead of several joins.

3. JSON Documents

3.1 Product Collection

Two products from different categories, showing the flexible schema.

```
{
  "_id": "P101",
  "name": "Galaxy M34 5G",
  "price": 18999.00,
  "currency": "INR",
  "stock_quantity": 120,
  "category": { "category_id": "C03", "name": "Mobiles" },
  "suppliers": [
    { "supplier_id": "S01", "name": "TechSource Pvt Ltd", "supply_price": 15500.00 },
    { "supplier_id": "S04", "name": "Mobile Hub", "supply_price": 15900.00 }
  ],
  "attributes": {
    "brand": "Samsung",
    "ram_gb": 6,
    "storage_gb": 128,
    "battery_mah": 6000,
    "color": ["Blue", "Silver"]
  },
  "tags": ["5g", "smartphone", "amoled"],
  "rating_summary": { "average": 4.3, "count": 2 },
  "is_active": true,
  "created_at": "2026-01-15T10:30:00Z"
}
```

```
{
  "_id": "P245",
  "name": "Cotton Round-Neck T-Shirt",
```

```

"price": 499.00,
"currency": "INR",
"stock_quantity": 340,
"category": { "category_id": "C07", "name": "Clothing" },
"suppliers": [
  { "supplier_id": "S09", "name": "Weave Textiles", "supply_price": 280.00 }
],
"attributes": {
  "brand": "UrbanFit",
  "fabric": "100% Cotton",
  "sizes": ["S", "M", "L", "XL"],
  "color": "Black"
},
"tags": ["tshirt", "casual"],
"rating_summary": { "average": 0, "count": 0 },
"is_active": true,
"created_at": "2026-02-02T08:00:00Z"
}

```

The `attributes` block differs between the two documents. That is the flexible schema in practice.

3.2 Category Collection

```

{
  "_id": "C03",
  "name": "Mobiles",
  "description": "Smartphones and accessories",
  "parent_category_id": "C01",
  "ancestors": ["C01"]
}

```

`parent_category_id` allows nested categories (Electronics > Mobiles), which the relational design did not support.

3.3 Review Collection

```

{
  "_id": "R5001",
  "product_id": "P101",
  "customer": { "customer_id": "CU017", "name": "Anitha R" },
  "rating": 5,
  "comments": "Excellent battery life.",
  "review_date": "2026-03-10T14:22:00Z"
}

```

3.4 Order Document (Optional Extension)

Orders are naturally one document because items are always read together with the order.

```

{
  "_id": "O9001",
  "customer_id": "CU017",
  "order_date": "2026-03-01T09:15:00Z",
  "order_status": "Delivered",
  "items": [
    { "product_id": "P101", "name": "Galaxy M34 5G", "quantity": 1, "unit_price": 18999.00 }
  ]
}

```

```

    ],
    "total_amount": 18999.00,
    "payment": { "method": "UPI", "status": "Paid", "date": "2026-03-01T09:16:00Z" },
    "shipment": { "status": "Delivered", "address_id": "A33" }
}

```

The `unit_price` is copied into the order. This is the same price-at-time-of-purchase principle as `Order_Details.unit_price` in the relational design.

4. Embed or Reference?

Data	Choice	Reason
Category name inside product	Embed (name + id)	Shown on every product page, rarely changes
Suppliers inside product	Embed	Few per product, read with the product
Rating summary inside product	Embed	Avoids computing the average on every page view
Reviews	Separate collection (reference <code>product_id</code>)	Can grow without limit; MongoDB documents have a 16 MB size cap
Order items inside order	Embed	Always read together
Customer details in order	Reference <code>customer_id</code> only	Customer data changes and is large

Rule of thumb: embed data that is read together and is bounded in size, and reference data that grows or is shared widely.

5. Key-Value Structures

Key-value stores (Redis, DynamoDB) provide very fast lookups by a single key, which suits hot data such as caches, stock counters, carts and sessions.

Key pattern	Value type	Example
<code>product:{id}</code>	JSON string (cached product)	<code>product:P101 = {"name": "Galaxy M34 5G", "price": 18999}</code>
<code>product:{id}:stock</code>	Integer counter	<code>product:P101:stock = 120</code>
<code>product:{id}:price</code>	Number	<code>product:P101:price = 18999</code>
<code>category:{id}:products</code>	Set of product IDs	<code>category:C03:products = {P101, P102, P110}</code>

Key pattern	Value type	Example
product:{id}:rating	Hash	product:P101:rating = {avg: 4.3, count: 2}
cart:{customer_id}	Hash of product to quantity	cart:CU017 = {P101: 1, P245: 2}
session:{token}	String with expiry (TTL)	session:a8f3... = CU017

Typical operations:

```
GET product:P101:stock
DECR product:P101:stock      # atomic stock reduction on purchase
SMEMBERS category:C03:products # list all products in a category
HSET cart:CU017 P101 1      # add to cart
EXPIRE session:a8f3 1800    # session expires in 30 minutes
```

DECR is atomic, so two customers buying the last item at the same moment cannot both succeed. This is a strong argument for using a key-value store for inventory counters.

6. Mapping from the Relational Design

Relational (Week 6)	NoSQL
PRODUCT row	products document
CATEGORY row	categories document (summary embedded in product)
PRODUCT_SUPPLIER join table	suppliers array inside product
REVIEW row	reviews document
ORDERS + ORDER_DETAILS	one orders document with items array
PRODUCT.stock_quantity	also cached as product:{id}:stock

7. Trade-offs

1. **Duplication.** If a category is renamed, every product embedding it must be updated. This is the cost of avoiding joins.
2. **Consistency.** Cached values (rating_summary, key-value copies) may be briefly out of date. This is the "eventual consistency" row of the SQL vs NoSQL comparison.
3. **Best fit.** Keep orders, payments and the inventory source of truth in SQL, where ACID transactions matter. Use NoSQL for the browsable catalog, reviews, carts and sessions.
4. **Indexes.** Create indexes on category.category_id, tags and price for fast filtering.